Trees

CMPT 145

Learning Objectives

After studying this chapter, a student should be able to:

- Describe what a tree is, in terms of its basic organizational principles.
- Explain the terms: node, child, parent, ancestor, descendant, sibling.
- Describe what a binary tree is, in terms of its basic organizational principle.
- Describe basic operations of binary tree ADT.

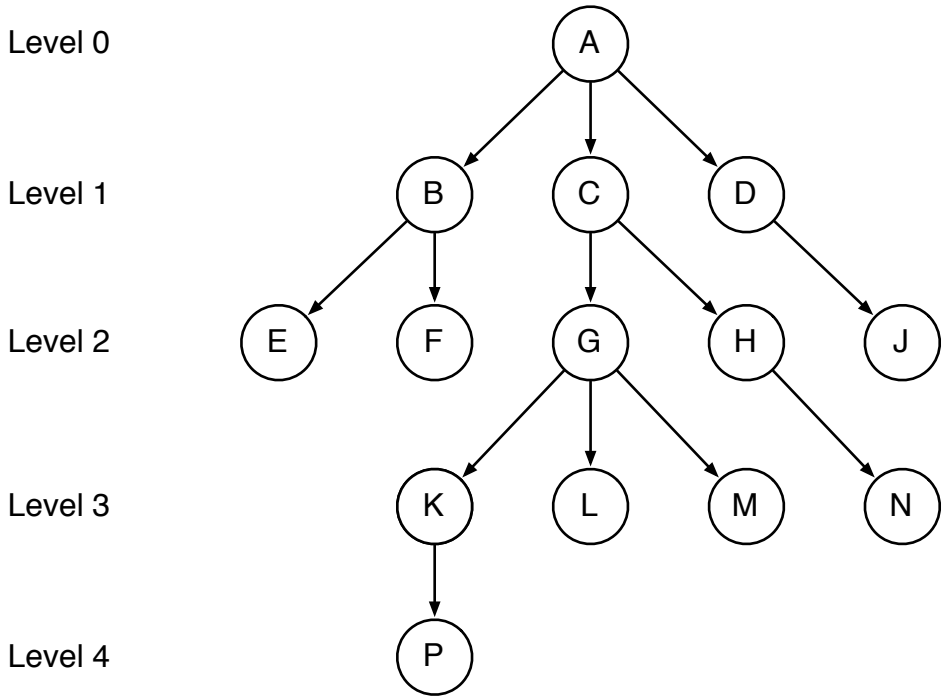
Trees organize data hierarchically

- Hierarchical organizations are common in companies, governments, etc.
- Textbooks are hierarchical: chapters, sections, paragraphs, etc.
- Computer file-systems are hierarchical: folders, sub-folders, documents.
- Computer application menus are hierarchical: menus, sub-menus.
- Hierarchies allow organization of activities!

Informal Terminology

- Data is stored in nodes.
- Nodes are connected with branches (a.k.a., edges, arcs, arrows)
- A branch connects a parent node to a child node.
- A parent may have 1 or more children; a child has exactly one parent.
- One node in a tree has no parents: the root
- Any node in the tree with no children is called a leaf node.

Example



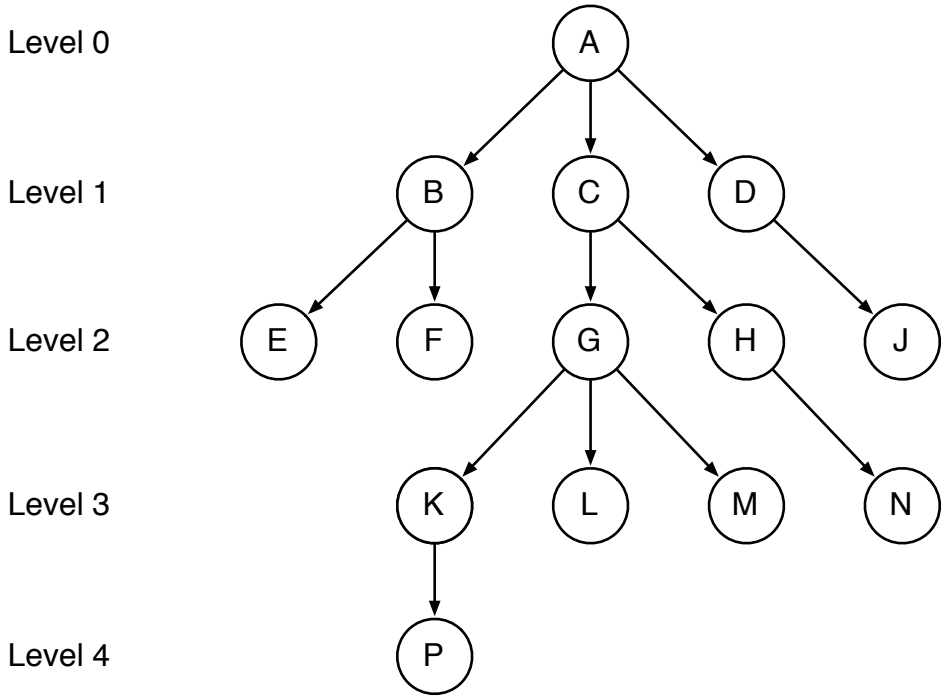
Root? Leaf? Parent? Child?

Formal Definition

A tree can be defined as follows:

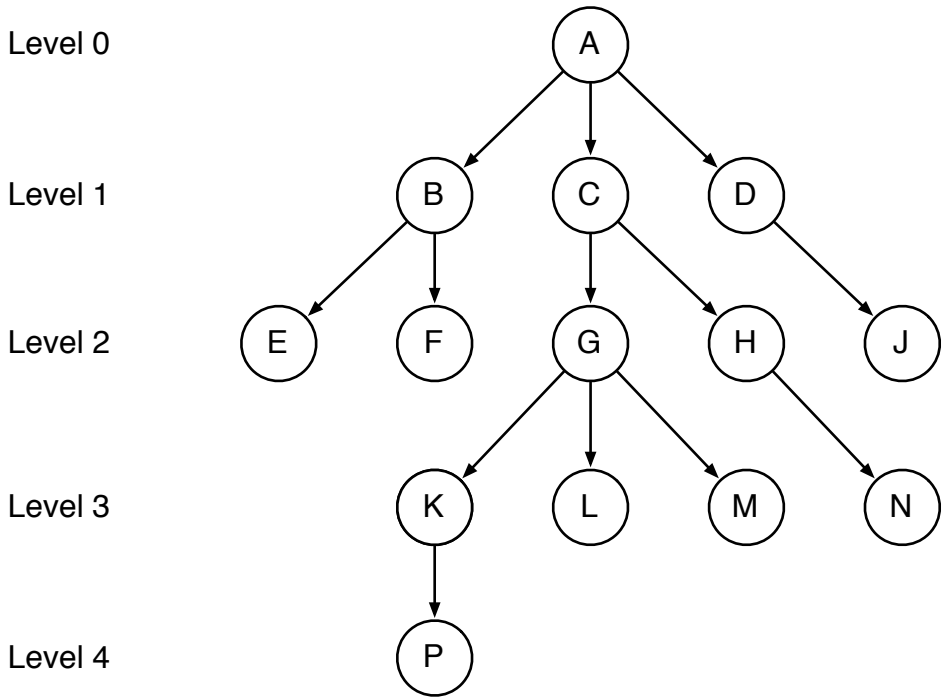
1. A structure with exactly zero nodes is an empty tree.
2. A structure with exactly one node is a tree.
3. If $t_1, \ldots, t_k$ are non-empty trees, then the structure, s, whose children are the roots of $t_1, \ldots, t_k$ is also a tree.

Definition: Path



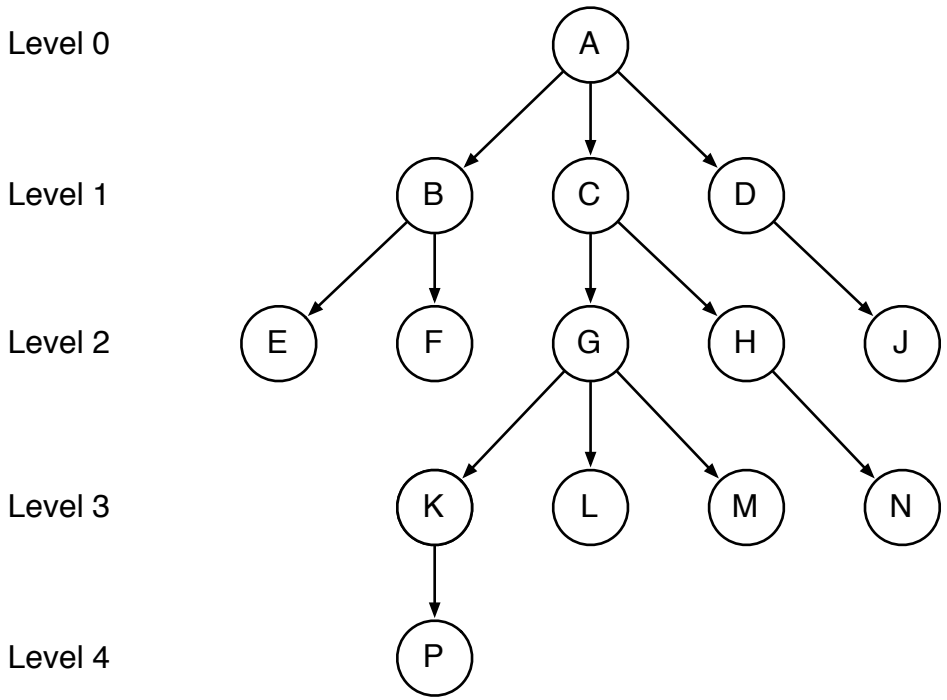
A path is a sequence of nodes connected by branches, with no repeated branches allowed.

Definition: Path length



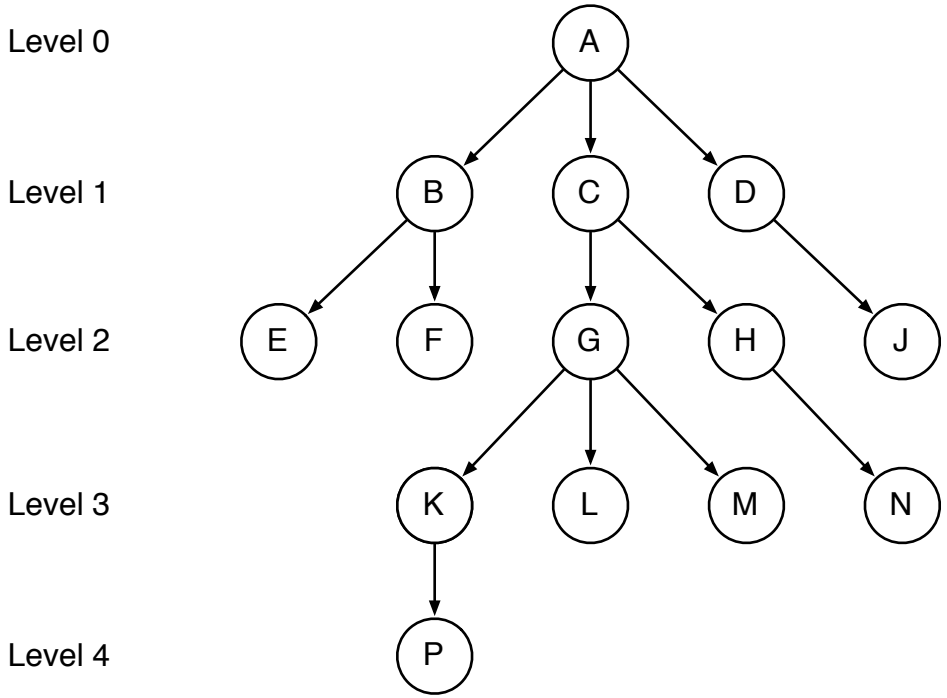
The length of the path is the number of branches in the path.

Definition: Level



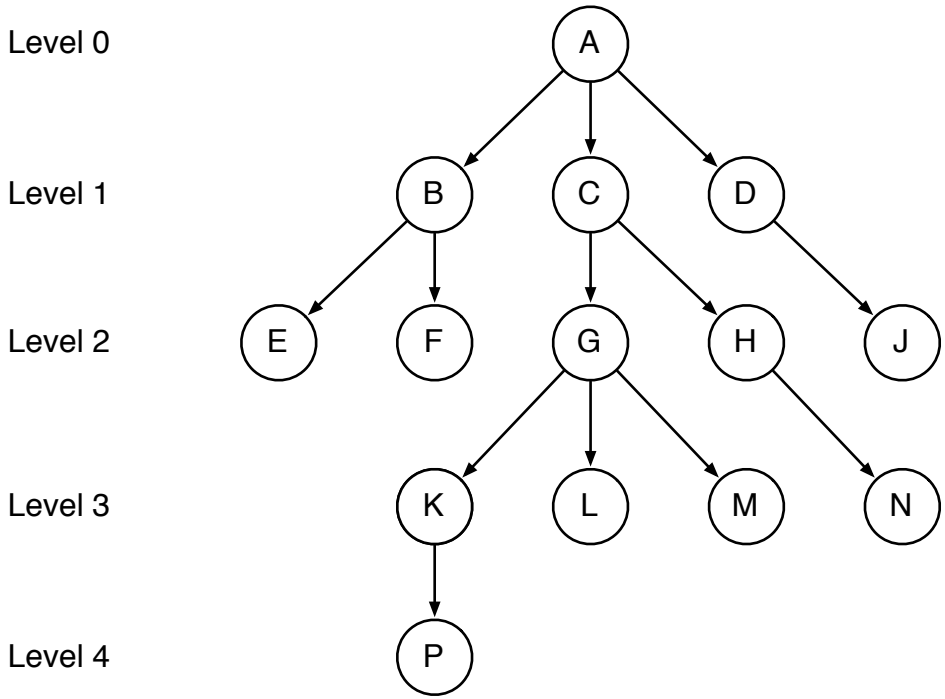
The term level describes where a node is in terms of the length of the path from the root to the node.

Definition: Height



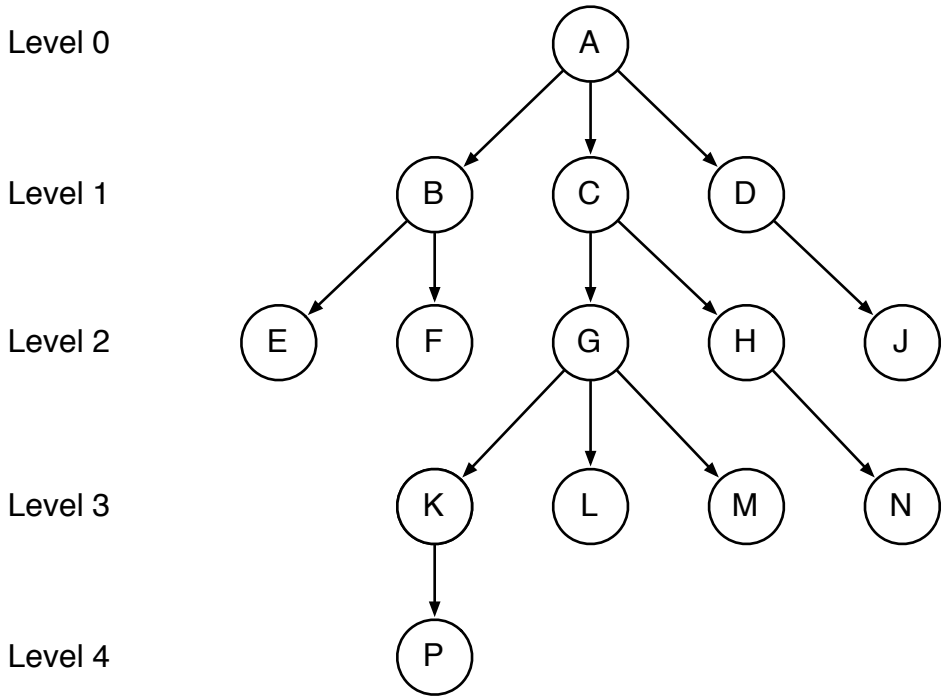
The height of a tree is always one more than the maximum level of any node in the tree.

Definition: Ancestors



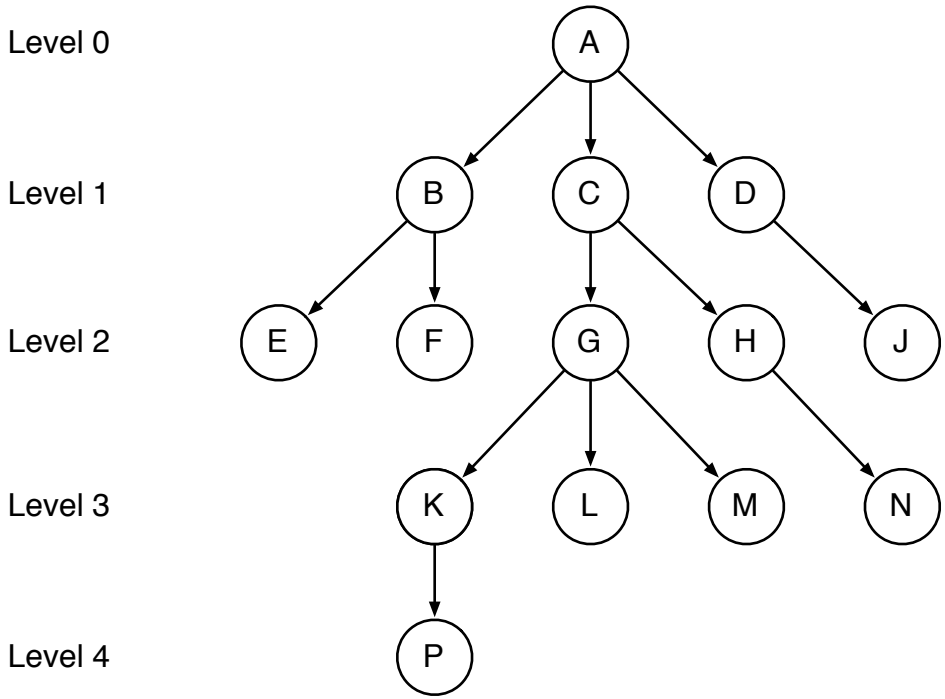
The ancestors of a node are nodes on the path from the node to the root.

Definition: Descendants



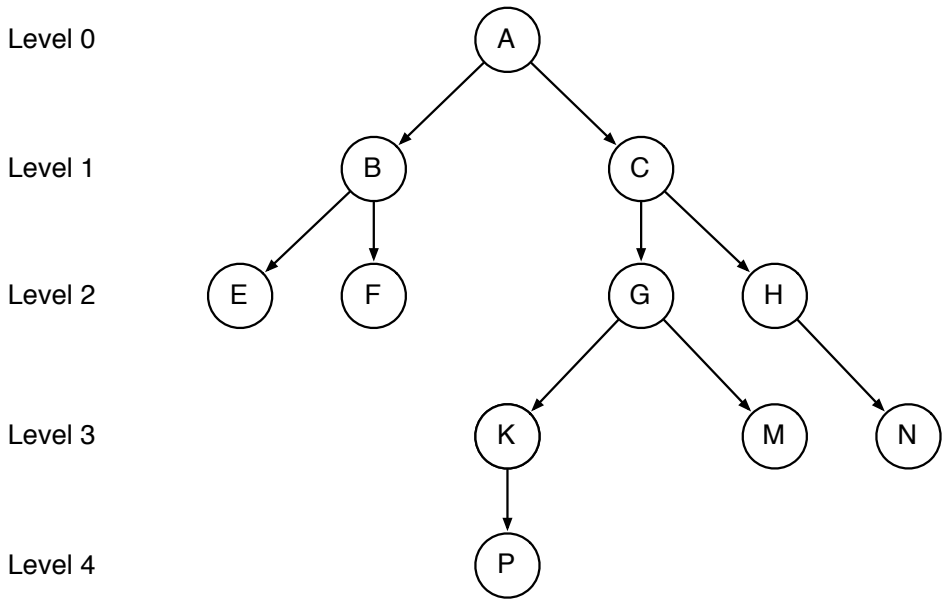
A node u is the descendant of a node v if v is an ancestor of u.

Definition: Sub-trees



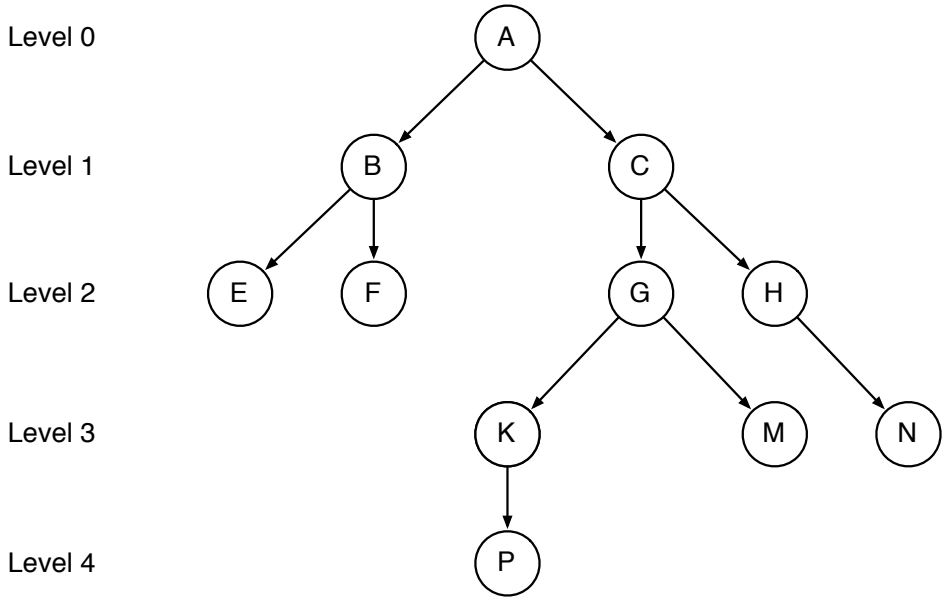
If u is a node in a tree, the sub-tree rooted at u consists of u and all of its descendants.

Definition: Binary trees



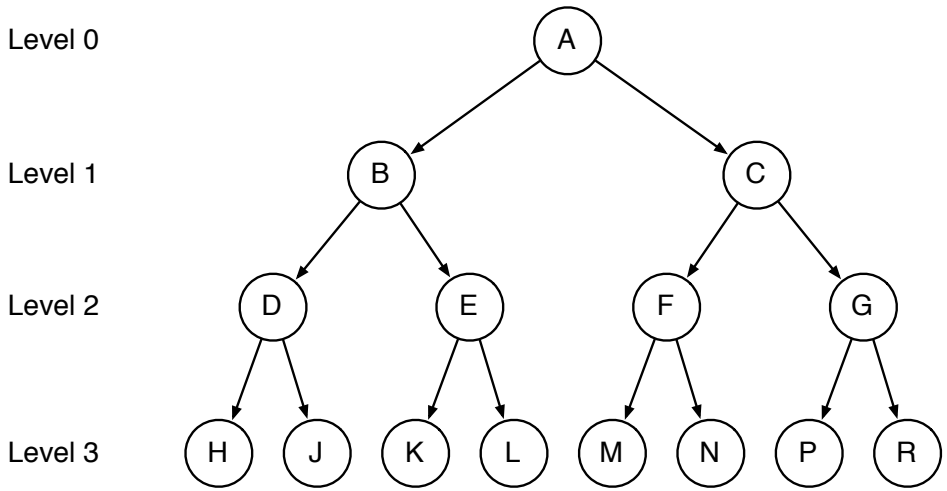
A binary tree is a special kind of tree with no node having more than 2 children.

Definition: Left and right sub-trees



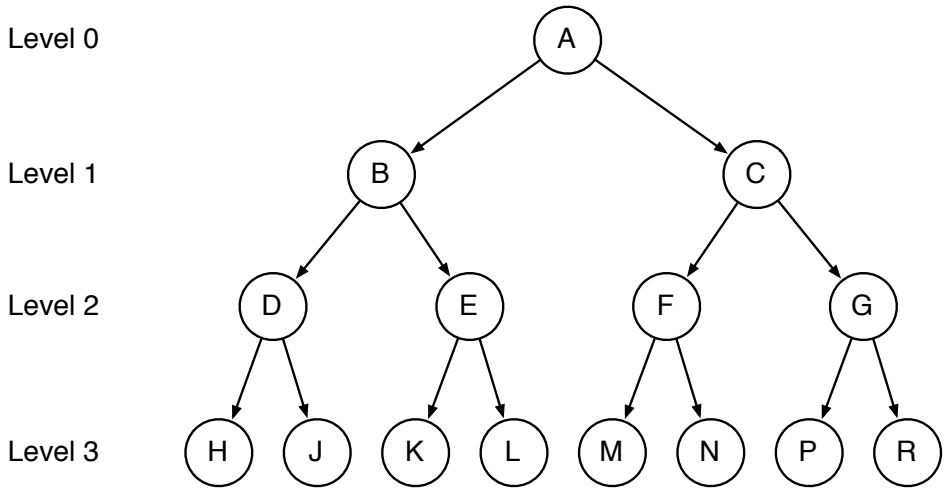
Since there are at most 2 children, we can label them left and right. Sometimes we say left sub-tree, or left node.

Definition: Complete Binary trees



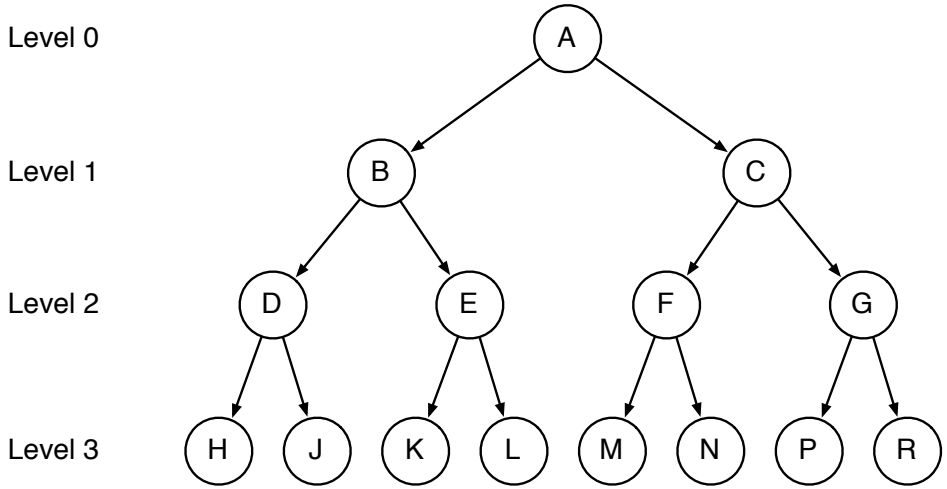
A complete binary tree is a binary tree that has exactly two children for every node, except for nodes at the maximum level, where the nodes are leaf nodes.

Property: Complete Binary tree levels



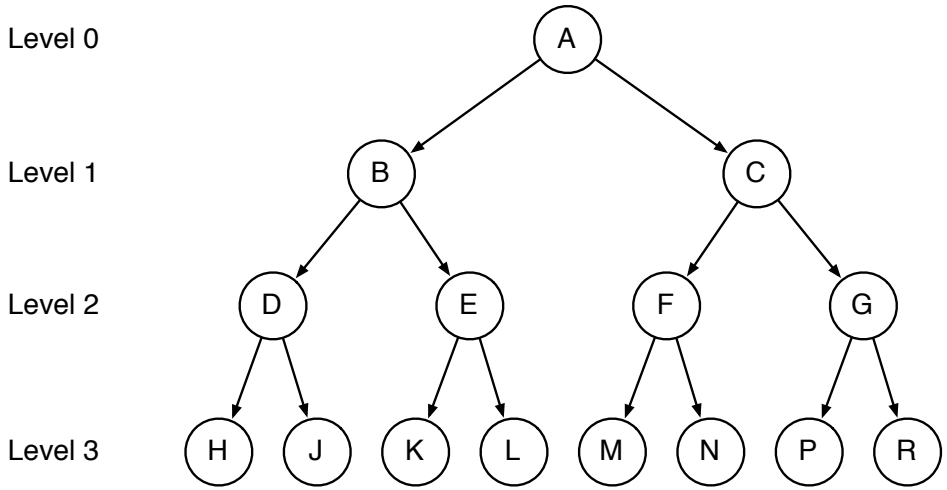
A complete binary tree has 2^l nodes at level l.

Property: Complete Binary tree height



In total, a complete binary tree whose height is h contains $2^h - 1$ nodes.

Property: Complete Binary tree height



A complete binary tree with n nodes has height $\log_2(n+1)$

Treenode Data Structure

A `treenode` is a simple object with three attributes:

data A data value
left A `treenode` (or the value `None`)
right A `treenode` (or the value `None`)

Treenode ADT

The primary operations provided for the `TreeNode` ADTs are as follows:

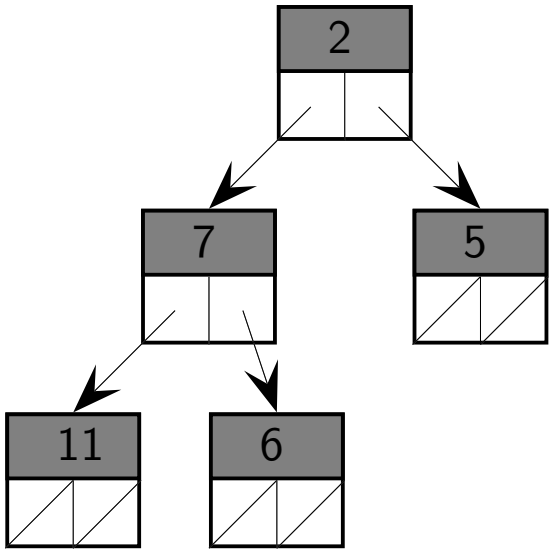
- `TreeNode(data, left=None, right=None)` creates a `TreeNode` object to contain the data value and the given left and right values. If left and right are not given, the value `None` is used by default.
- `treenode.get_data()` returns the contents of the data attribute of the given `treenode`
- `treenode.get_left()` returns the contents of the left attribute of the given `treenode`
- `treenode.get_right()` returns the contents of the right attribute of the given `treenode`

Treenode ADT

The primary operations provided for the `treenode` ADTs are as follows:

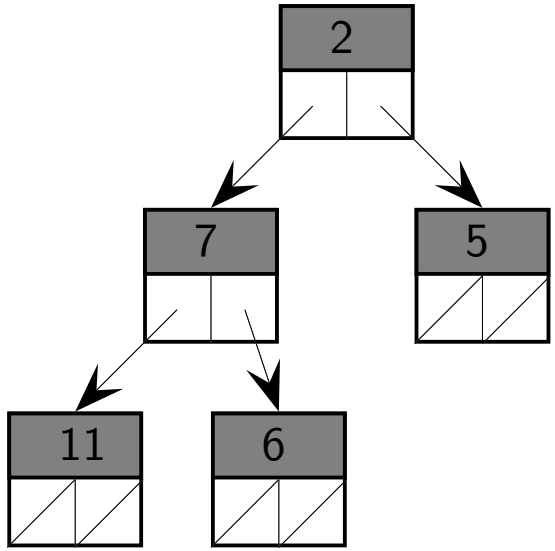
- `treenode.set_data(v)` given a `treenode`, sets the contents of the data attribute to `v`
- `treenode.set_left(n)` given a `treenode`, sets the contents of the left attribute to `n`
- `treenode.set_right(n)` given a `treenode`, sets the contents of the right attribute to `n`

Exercise



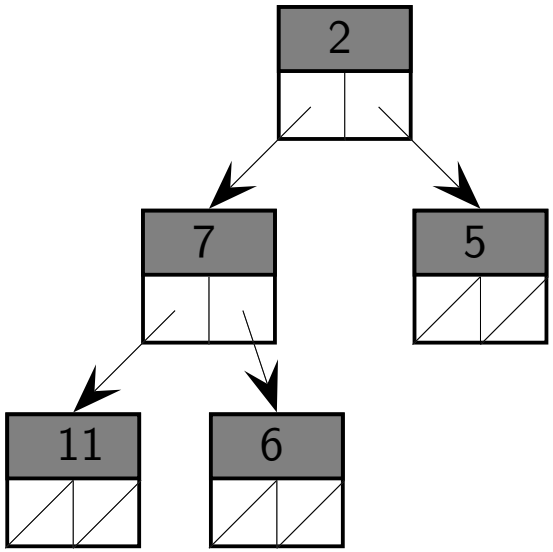
Use the treenode ADT to construct the given binary tree.

Exercise



Add a new left child to node 5.

Exercise



Add a new right child to node 6.